

Ex-no: 9	BANKER'S ALGORITHM
/03/2026	

Aim

To study and implement the Banker's Algorithm for deadlock avoidance and to determine whether the system is in a safe state.

Algorithm

1. Initialize the following matrices:
 - o Allocation matrix
 - o Maximum matrix
 - o Available resources vector
2. Calculate the Need matrix using:
Need = Max – Allocation
3. Initialize:
 - o Work = Available
 - o Finish[i] = false for all processes
4. Find a process i such that:
 - o Finish[i] == false
 - o Need[i] ≤ Work
5. If such a process is found:
 - o Add its allocation to Work → Work = Work + Allocation[i]
 - o Mark Finish[i] = true
 - o Add process i to the safe sequence
6. Repeat steps 4–5 until all processes are marked finished or no such process is found.
7. If all Finish[i] == true, the system is in a **safe state**; otherwise, it is **unsafe**.

CODE:

```
// BANKERS ALGO
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int p, r, i, j, k;
```

```
    printf("Enter number of processes: \n");
```

```
    scanf("%d", &p);
```

```
printf("Enter number of resource types: \n");
```

```
scanf("%d", &r);
```

```
int alloc[p][r], max[p][r], need[p][r];
```

```
int avail[r], work[r];
```

```
int finish[p], safeSeq[p];
```

```
printf("Enter Allocation Matrix:\n");
```

```
for(i = 0; i < p; i++)
```

```
    for(j = 0; j < r; j++)
```

```
        scanf("%d", &alloc[i][j]);
```

```
printf("Enter Max Matrix:\n");
```

```
for(i = 0; i < p; i++)
```

```
    for(j = 0; j < r; j++)
```

```
        scanf("%d", &max[i][j]);
```

```
printf("Enter Available Resources: \n");
```

```
for(i = 0; i < r; i++) {
```

```
    scanf("%d", &avail[i]);
```

```
    work[i] = avail[i];
```

```
}
```

```
for(i = 0; i < p; i++)
```

```
    for(j = 0; j < r; j++)
```

```
        need[i][j] = max[i][j] - alloc[i][j];
```

```
for(i = 0; i < p; i++)
```

```
    finish[i] = 0;
```

```
int count = 0;
```

```

while(count < p) {

    int found = 0;

    for(i = 0; i < p; i++) {

        if(finish[i] == 0) {

            int possible = 1;

            for(j = 0; j < r; j++) {

                if(need[i][j] > work[j]) {

                    possible = 0;

                    break;

                }

            }

            if(possible) {

                for(k = 0; k < r; k++)

                    work[k] += alloc[i][k];

                safeSeq[count++] = i;

                finish[i] = 1;

                found = 1;

            }

        }

    }

}

```

```

        if(!found)

            break;
    }

    if(count == p) {

        printf("System is in a SAFE STATE.\n");

        printf("Safe Sequence: ");

        for(i = 0; i < p; i++)

            printf("P%d ", safeSeq[i]);

    }

    else {

        printf("System is NOT in a safe state (Deadlock may occur).\n");

    }


    return 0;
}

```

OUTPUT:

Sample Input

```

Enter number of processes: 5
Enter number of resource types: 3

```

Allocation Matrix

```

0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

```

Max Matrix

7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Available Resources

3 3 2

System is in a SAFE STATE.

Safe Sequence: P1 P3 P4 P0 P2

Result

The Banker's Algorithm was successfully implemented. The system's safety was determined, and the safe sequence of processes was identified, demonstrating deadlock avoidance in resource allocation.